

Megh Patel, Ellie Williams, Nathan McClure

Professor Radian

CSC-2400 Design of Algorithms

4/26/26

Clustering Optimization

Overview

The problem of cluster optimization is partitioning a dataset into k clusters while keeping Within-Cluster Sum of Squares (WCSS) as low as possible. This means a dataset is grouped into k clusters with the goal of having each cluster contain stuff that is as close to the average (center) of that cluster. It is NP-hard as it takes exponentially larger amounts of time to find optimal cluster groups. The algorithms we experimented with were K-means, K-medoids, Hierarchical Agglomerative (bottom-up) Clustering, and Particle Swarm Optimization.

Objective / Research question

- i. **Question 1:** How does runtime and memory usage of K-means, K-medoids, and Hierarchical Agglomerative Clustering scale as input n increases?

K-means grows in linear time making it the most efficient as it grows. K-medoids grows quadratically meaning it takes longer to execute, but its results are better than K-means. HAC is the slowest of the three as it scales. The biggest tradeoff is how quickly you want results or how thorough you want the results to be.
- ii. **Question 2:** For fixed input size, how do K-means, k-medoids, and PSO-based clustering trade off solution quality versus runtime as the number of clusters k increases?

K-means is the fastest but gives the worst result. K-medoids is slower but gives a better result. Lastly PSO is the slowest but gives the most thorough result. The main trade off is the time spent waiting and the quality of the result.

- iii. **Question 3:** For PSO-based clustering, what is the trade-off between solution quality and computational cost as PSO parameters change?

Refining the parameters has a linear relationship with computational cost. This means any increase in parameters is going to directly increase the computational cost of running the algorithm.

Experiment Description

We coded all the actual algorithms in c++, and we used Python for graphing and visualizing our data. We used two datasets for experimentation, one synthetic and one real. Our real dataset was a csv of mall customer data and was very small, only around 200 points. Our real dataset was scikit's blob generator that we used ai to translate to a c++ equivalent. The synthetic set was tested for exponentially more points; we tested 100, 200, 400, 800, and 1600 points. We repeated each trial 5 times and averaged the data. We ran our algorithms on our respective machines, but all of them have pretty comparable specs. HAC is a greedy algorithm, K-means and K-medoids are heuristic, and PSO is a heuristic algorithm. Each member generated and ran their own algorithm and performed their own trial runs. We all used the same synthetic data generator but not the same generated synthetic data. All algorithms ran at about the expected runtimes. HAC's worst case can be as bad as $O(n^3)$ and it was evident during experimentation. K-means was an extremely efficient algorithm with each run tested lasting less than a 100th of a second. K-means runs in linear time as well, making it the most efficient in theory as well. K-medoids also ran well but nowhere near as efficiently. This aligns with its quadratic runtime. All K-medoids runs averaged .25-.4 seconds for 1600 clusters. For the PSO

algorithm, a standard swarm-based clustering approach was implemented. The swarm size and iteration count was kept fixed across all tests to ensure consistency. Since PSO maintains and updates a full population of candidate solutions, its runtime naturally scales with both the dataset size and the number of particles. In our experiments, PSO performed efficiently on smaller datasets and showed a smooth, roughly linear increase in runtime as n increased, which matches its expected $O(n \times \text{swarm size} \times \text{iterations})$ behavior.

Results (Graphs on Last Page)

Our results pretty much matched what we had researched prior to experimentation. HAC performed worse than we anticipated, according to our empirical analysis it performed closer to $O(n^3)$ than $O(n^2 \log n)$ like our research suggested. This could have been from a multitude of reasons such as the computer it was ran on or our codes overall efficiency. It did raise the question of when HAC is actually useful? It is outperformed in every test by all the algorithms we experimented with. For HAC, the main limitation was time, its runtime grew so exponentially fast that it was not feasible to attempt any large datasets. Additionally if I was given more time and the chance to do this project all over again, I would've liked to generate multiple of the same algorithm to potentially find a version that runs more effectively. A huge limitation to the efficiency comparison of K-means and K-medoids is the limit in data sets. Our data set had to be small to ensure HAC could run, but k-means and k-medoids need larger data sets to get a clearer comparison. The smaller scale tests still show the different growth rates, but a larger scale would make the difference more visible in the graphs. If I redid this assignment, I would use a separate larger dataset for K-means and K-medoids to better show how K-medoids' quadratic efficiency worsens the larger the dataset is. For PSO, our empirical analysis focused on the trade-off between solution quality and computational cost as swarm size and iteration count changed. Even though PSO was running on smaller datasets due to HAC's

limitations, the trends we observed still reflected the behavior expected in our larger research question. Increasing the swarm size or number of iterations improved solution quality (lower WCSS), but it also raised runtime because more particles and more updates must be processed each iteration. We also saw diminishing returns, where larger parameter values produced only small quality gains while runtime continued to grow almost linearly. If I could do this again with more time, I would've run PSO on much larger datasets. This raises the question, how quickly does PSO's solution quality plateau as parameters increase, and is there an optimal swarm size for given dataset dimension.

Sources Used

- iv. <https://www.geeksforgeeks.org/machine-learning/k-means-vs-k-medoids-clustering/>
- v. <https://www.geeksforgeeks.org/machine-learning/k-medoids-clustering-in-machine-learning/>
- vi. <https://www.geeksforgeeks.org/machine-learning/k-means-clustering-introduction/>
- vii. <https://www.meta-chart.com>
- viii. <https://www.kaggle.com/datasets/shwetabh123/mall-customers>
- ix. Copilot and ChatGPT were used to assist in the coding process of the algorithms

Work Split

Task:	Ellie:	Nathan:	Megh:
-------	--------	---------	-------

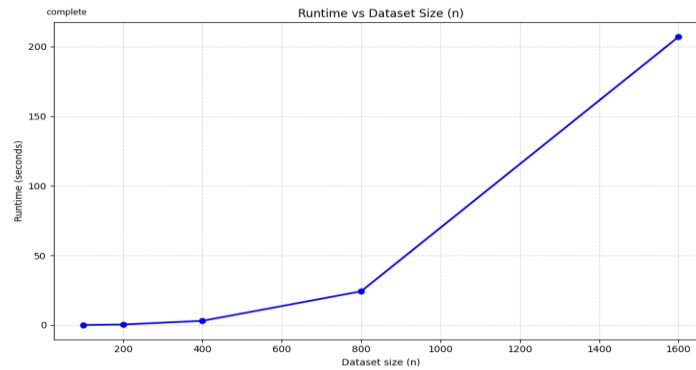
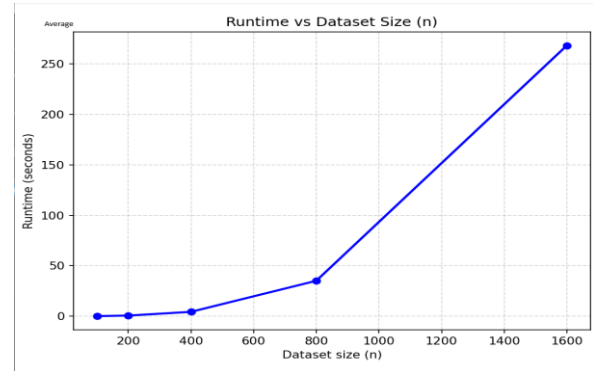
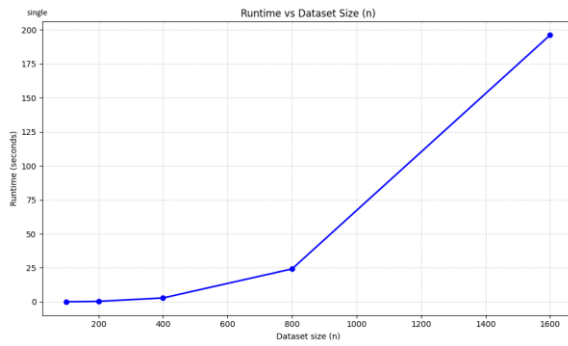
1 st Checkpoint	N/A	Overview, Research Q's, experimental design, communication, alg selection	Research Q's, experimental design, goals, risk assessment, dataset, alg selection
Code	k-means, k-medoids	HAC, cleaning the Mall data set.	PSO
slides	Design, K-means, K-medoids	Hac content	PSO content
Report	Results, Description of Experiments, Sources	Results, Description of Experiments, Overview	Results, Description of Experiments, Graphs

Roadblocks

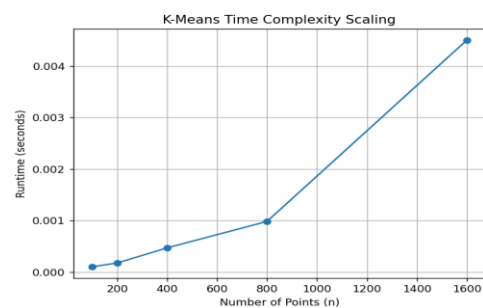
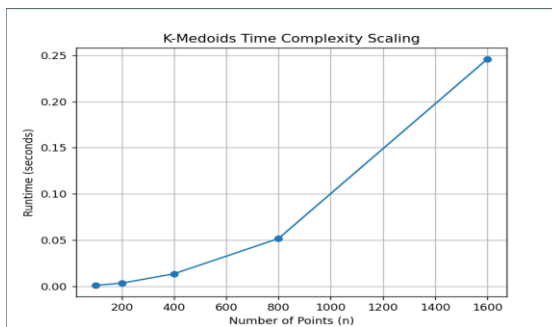
The main difficulty was balancing the work for this project with our other classes and schedules. Originally, we had much more ambitious plans for the project but were unable to them with how busy our schedules became. We also had a change in group members after the first checkpoint, which meant we had to re-allocate work and algorithm assignments. And while adding an extra person was helpful to lighten the workload, it was difficult at first to add a new person into the flow so late in the semester.

Annex

- HAC



- K-medoids and K-means



- PSO

